

Train and evaluate models with tidymodels

2024-02-15

This template offers an opinionated guide on how to structure a modeling analysis. Your individual modeling analysis may require you to add to, subtract from, or otherwise change this structure, but consider this a general framework to start from. If you want to learn more about using tidymodels, check out our Getting Started (<https://www.tidymodels.org/start/>) guide.

In this example analysis, let's fit a model to predict the sex of penguins (<https://allisonhorst.github.io/palmerpenguins/>) from species and measurement information.

```
library(tidymodels)
```

```
## Warning: package 'tidymodels' was built under R version 4.2.3
```

```
## — Attaching packages ————— tidymodels 1.1.0 —
```

```
## ✓ broom          1.0.4      ✓ recipes          1.0.6
## ✓ dials          1.2.0      ✓ rsample          1.1.1
## ✓ dplyr          1.1.3      ✓ tibble           3.2.1
## ✓ ggplot2        3.4.2      ✓ tidyr            1.3.0
## ✓ infer          1.0.4      ✓ tune             1.1.1
## ✓ modeldata      1.2.0      ✓ workflows        1.1.3
## ✓ parsnip        1.1.0      ✓ workflowsets     1.0.1
## ✓ purrr          1.0.1      ✓ yardstick        1.2.0
```

```
## Warning: package 'broom' was built under R version 4.2.3
```

```
## Warning: package 'dials' was built under R version 4.2.3
```

```
## Warning: package 'scales' was built under R version 4.2.3
```

```
## Warning: package 'dplyr' was built under R version 4.2.3
```

```
## Warning: package 'ggplot2' was built under R version 4.2.3
```

```
## Warning: package 'infer' was built under R version 4.2.3
```

```
## Warning: package 'parsnip' was built under R version 4.2.3
```

```
## Warning: package 'purrr' was built under R version 4.2.3
```

```
## Warning: package 'recipes' was built under R version 4.2.3
```

```
## Warning: package 'rsample' was built under R version 4.2.3
```

```
## Warning: package 'tibble' was built under R version 4.2.3
```

```
## Warning: package 'tidyr' was built under R version 4.2.3
```

```
## Warning: package 'tune' was built under R version 4.2.3
```

```
## Warning: package 'workflows' was built under R version 4.2.3
```

```
## Warning: package 'workflowsets' was built under R version 4.2.3
```

```
## Warning: package 'yardstick' was built under R version 4.2.3
```

```
## — Conflicts ————— tidymodels_conflicts() —  
## ✖ purrr::discard() masks scales::discard()  
## ✖ dplyr::filter()   masks stats::filter()  
## ✖ dplyr::lag()      masks stats::lag()  
## ✖ recipes::step()  masks stats::step()  
## • Dig deeper into tidy modeling with R at https://www.tmw.r.org
```

```
data(penguins)  
glimpse(penguins)
```

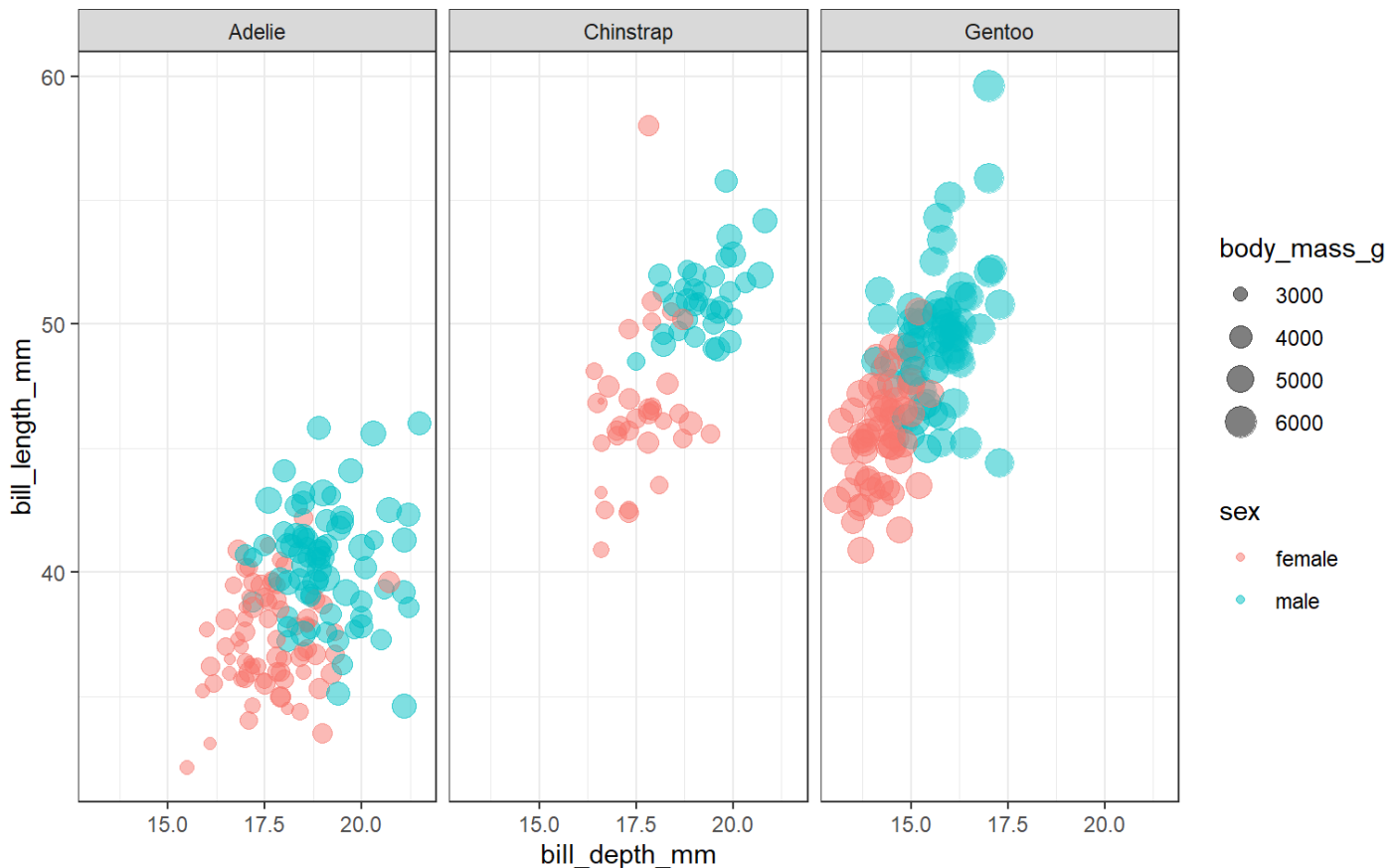
```
## Rows: 344  
## Columns: 7  
## $ species      <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel...  
## $ island        <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgersen, Torgersen...  
## $ bill_length_mm <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, ...  
## $ bill_depth_mm <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, ...  
## $ flipper_length_mm <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186...  
## $ body_mass_g    <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, ...  
## $ sex            <fct> male, female, female, NA, female, male, female, male...
```

```
penguins <- na.omit(penguins)
```

Explore data

Exploratory data analysis (EDA) is an important part of the modeling process (<https://www.tnwr.org/software-modeling.html#model-phases>).

```
penguins %>%  
  ggplot(aes(bill_depth_mm, bill_length_mm, color = sex, size = body_mass_g)) +  
  geom_point(alpha = 0.5) +  
  facet_wrap(~species) +  
  theme_bw()
```



Build models

Let's consider how to spend our data budget (<https://www.tnwr.org/splitting.html>):

- create training and testing sets
- create resampling folds from the *training* set

```
set.seed(123)  
penguin_split <- initial_split(penguins, strata = sex)  
penguin_train <- training(penguin_split)  
penguin_test <- testing(penguin_split)  
  
set.seed(234)  
penguin_folds <- vfold_cv(penguin_train, strata = sex)  
penguin_folds
```

```
## # 10-fold cross-validation using stratification
## # A tibble: 10 × 2
##   splits      id
##   <list>      <chr>
## 1 <split [223/26]> Fold01
## 2 <split [223/26]> Fold02
## 3 <split [223/26]> Fold03
## 4 <split [224/25]> Fold04
## 5 <split [224/25]> Fold05
## 6 <split [224/25]> Fold06
## 7 <split [225/24]> Fold07
## 8 <split [225/24]> Fold08
## 9 <split [225/24]> Fold09
## 10 <split [225/24]> Fold10
```

Let's create a **model specification** (<https://www.tmwr.org/models.html>) for each model we want to try:

```
glm_spec <-
  logistic_reg() %>%
  set_engine("glm")

ranger_spec <-
  rand_forest(trees = 1e3) %>%
  set_engine("ranger") %>%
  set_mode("classification")
```

To set up your modeling code, consider using the **parsnip** addin (https://parsnip.tidymodels.org/reference/parsnip_addin.html) or the **usemodels** (<https://usemodels.tidymodels.org/>) package.

Now let's build a **model workflow** (<https://www.tmwr.org/workflows.html>) combining each model specification with a data preprocessor:

```
penguin_formula <- sex ~ .

glm_wf <- workflow(penguin_formula, glm_spec)
ranger_wf <- workflow(penguin_formula, ranger_spec)
```

If your feature engineering needs are more complex than provided by a formula like `sex ~ .`, use a recipe (<https://www.tidymodels.org/start/recipes/>). Read more about feature engineering with recipes (<https://www.tmwr.org/recipes.html>) to learn how they work.

Evaluate models

These models have no tuning parameters so we can evaluate them as they are. Learn about tuning hyperparameters here. (<https://www.tidymodels.org/start/tuning/>)

```

contrl_preds <- control_resamples(save_pred = TRUE)

glm_rs <- fit_resamples(
  glm_wf,
  resamples = penguin_folds,
  control = contrl_preds
)

```

```
## → A | warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## There were issues with some computations    A: x1
## There were issues with some computations    A: x1
```

```
##
```

```

ranger_rs <- fit_resamples(
  ranger_wf,
  resamples = penguin_folds,
  control = contrl_preds
)

```

```
## Warning: package 'ranger' was built under R version 4.2.3
```

How did these two models compare?

```
collect_metrics(glm_rs)
```

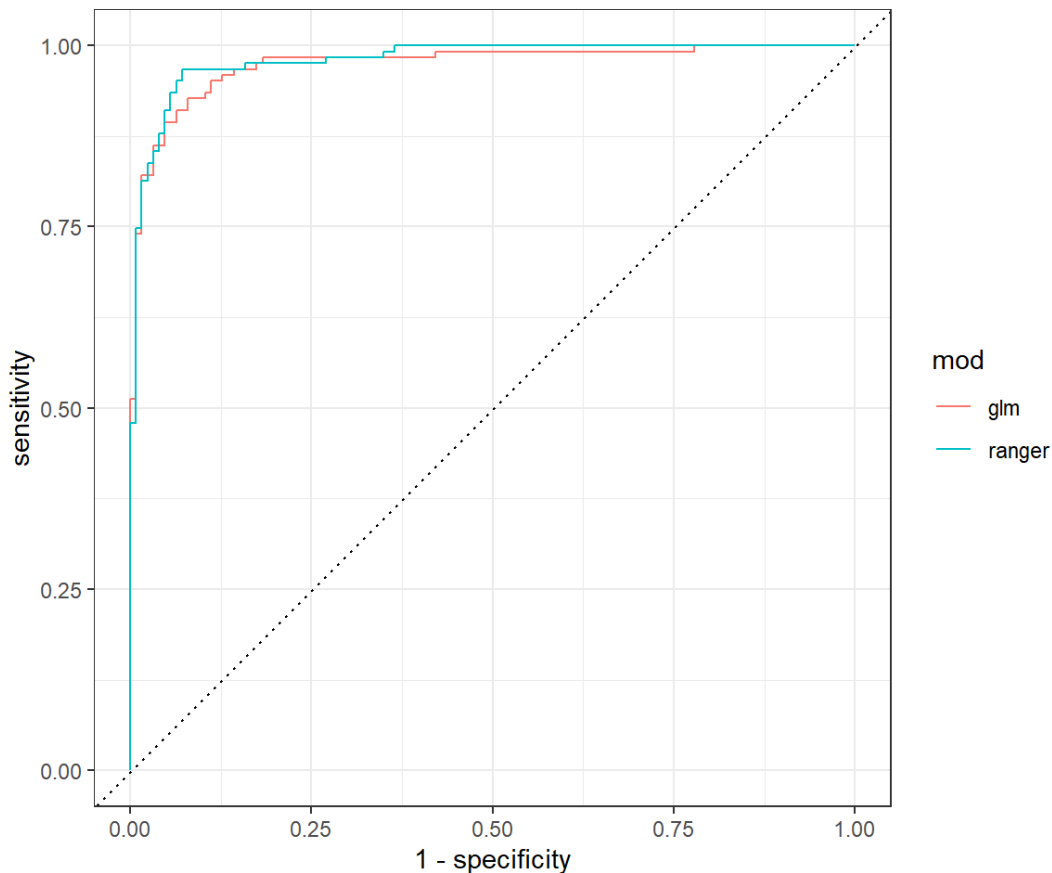
```
## # A tibble: 2 × 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy binary    0.916   10  0.0173 Preprocessor1_Model1
## 2 roc_auc  binary    0.975   10  0.0105 Preprocessor1_Model1
```

```
collect_metrics(ranger_rs)
```

```
## # A tibble: 2 × 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy binary    0.936   10  0.0146 Preprocessor1_Model1
## 2 roc_auc  binary    0.982   10  0.00805 Preprocessor1_Model1
```

We can visualize these results using an ROC curve (or a confusion matrix via `conf_mat()`):

```
bind_rows(
  collect_predictions(glm_rs) %>%
    mutate(mod = "glm"),
  collect_predictions(ranger_rs) %>%
    mutate(mod = "ranger")
) %>%
  group_by(mod) %>%
  roc_curve(sex, .pred_female) %>%
  autoplot()
```



These models perform very similarly, so perhaps we would choose the simpler, linear model. The function `last_fit()` *fits* one final time on the training data and *evaluates* on the testing data. This is the first time we have used the testing data.

```
final_fitted <- last_fit(glm_wf, penguin_split)
collect_metrics(final_fitted) ## metrics evaluated on the *testing* data
```

```
## # A tibble: 2 × 4
##   .metric .estimator .estimate .config
##   <chr>    <chr>         <dbl> <chr>
## 1 accuracy binary         0.857 Preprocessor1_Model1
## 2 roc_auc  binary         0.937 Preprocessor1_Model1
```

This object contains a fitted workflow that we can use for prediction.

```
final_wf <- extract_workflow(final_fitted)
predict(final_wf, penguin_test[55,])
```

```
## # A tibble: 1 × 1
##   .pred_class
##   <fct>
## 1 female
```

You can save this fitted `final_wf` object to use later with new data, for example with `readr::write_rds()` .